Workshop No. 2 – Kaggle Systems Engineering Analysis

UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Juan Pablo Arismendi Sánchez

Juliana Camila Rincón Rojas

Samuel Casas Cantor

April 25 ,2025

1. Introduction

Demand forecasting is a critical process in supply chain management, allowing organizations to optimize logistics, reduce waste, and improve customer satisfaction. This report presents a system design proposal based on the Kaggle competition "Grupo Bimbo Inventory Demand," where the objective is to build a predictive model capable of forecasting weekly product demand for Mexico's largest baked goods distribution network. The system design integrates the outcomes from a detailed systems analysis conducted in Workshop #1, addressing identified constraints, data complexities, and chaos-theory implications.

2. System Requirements Specification

2.1 Context and Problem Overview

Grupo Bimbo's distribution process involves delivering over 1,000 different products to over 1 million stores using a direct store delivery model. Each week, thousands of sales routes deliver products to points of sale, generating large volumes of transactional data. However, overproduction or underproduction leads to either waste or unmet customer demand.

The challenge is to design a data-driven forecasting system capable of:

- Processing massive historical sales data.
- Extracting actionable features.
- Training reliable machine learning models.
- Generating accurate forecasts in a reproducible, scalable, and maintainable manner.

2.2 Functional Requirements

2.2.1 Data Ingestion

The system must support automated ingestion of large datasets provided in tabular formats (CSV), including:

- **Historical Sales Data**: Transactional records including `Week`, `Client_ID`, `Product_ID`, and `Sales Units`.

- **Product Metadata**: Product identifiers and descriptions.
- **Client Metadata**: Client identifiers, geographic segmentation, and store types.
- **Channel Information**: Data related to sales channels such as supermarkets, convenience stores, and small retailers.

2.2.2 Data Preprocessing and Transformation

Once the data is loaded, the system must:

- Validate schema and check data integrity.
- Handle missing values, duplicates, or outliers.
- Aggregate demand metrics at different levels (e.g., Client-Product-Week).
- Transform target variables using logarithmic scaling to stabilize variance.

2.2.3 Feature Engineering

To enrich model inputs, the system must generate:

- **Lagged Features**: Aggregates (mean, sum, min, max) over **1-3 week rolling windows** to capture historical demand patterns.
- **Text-Derived Product Features**: Extracted from **product names**, including weight, brand, and product family identifiers.
- **Cluster-Based Features**: Group products into **behaviorally similar clusters** to generalize from limited data.
- **Frequency Features**: Count occurrences of client-product combinations to capture **transactional regularity**.
- **Median-Based Backup Predictions**: Use **historical medians** for sparse client-product pairs to improve **forecast stability** in low-data scenarios.
  (Highlighted by Lahiru Madushanka's report.)
- **Feature Selection**: Apply **importance ranking** (e.g., `xgb.importance()`) to retain **top N features** (e.g., 175), balancing complexity and performance.

2.2.4 Model Training and Optimization

The system must implement supervised machine learning techniques such as XGBoost, known for handling large datasets and noisy environments effectively. The system must support **multi-level ensembling**, combining:

- **First-Level Models**: Multiple **XGBoost regressors**, trained with different feature sets and hyperparameters.
- **Second-Level Models**:

- **ExtraTrees Classifier** (robust to noise, captures non-linearities).
- **Linear Regression Model** (provides stability in blending).

Validation must simulate **operational forecasting**, predicting one week, recalculating features, and predicting the next.

2.2.5 Prediction and Submission Generation

The trained model must produce forecasts for unseen data and export them in Kaggle's specified submission format, ensuring all predictions are positive and realistic.

2.3 Non-Functional Requirements

2.3.1 Performance and Scalability

The system must process tens of millions of rows in a reasonable timeframe, leveraging parallel processing or distributed computing when necessary.

2.3.2 Modularity and Maintainability

Each system component (data ingestion, transformation, modeling, prediction) must be independently executable and replaceable without affecting the entire pipeline.

2.3.3 Reproducibility

Experiments must be fully reproducible through version-controlled code, fixed random seeds, and logged configurations.

2.3.4 Usability

The system must expose configuration files or command-line interfaces for users to define parameters such as dataset paths, model settings, and execution modes.

2.4 Sensitivity and Chaos Management

2.4.1 Sensitivity Management

The system must handle:

- Variations in data availability and quality.
- Client and product behavioral changes over time.
- Model degradation due to non-stationary data streams.

Techniques include:

- Dynamic feature recalculation.
- Periodic retraining with recent data.
- Regular performance monitoring.

2.4.2 Chaos Mitigation

Considering the potential for cascading errors in large-scale pipelines, the system must:

- Validate intermediate outputs at each processing stage.
- Log runtime events, warnings, and exceptions.
- Apply conservative thresholds to prevent error amplification.
- Provide rollback or retry mechanisms in case of failures.

2.5. Summary of Requirements

Category	Description
Data Sources	Historical Sales, Product Metadata, Client Metadata, Channel Data
Processing	Data validation, cleaning, aggregation, transformation
Features	Temporal patterns, client-product interactions, product/client attributes
Modeling	XGBoost-based regression, cross-validation, hyperparameter tuning
Evaluation Metric	RMSLE (Root Mean Squared Logarithmic Error)
Outputs	Kaggle-formatted submission file
Performance	Capable of processing tens of millions of records
Modularity	Independent components for ingestion, processing, modeling, and output
Reproducibility	Version-controlled, seed-controlled, and configuration-logged
Usability	Parameterized execution interfaces
Security	Data privacy and access control, especially on cloud platforms
Sensitivity Control	Dynamic recalculation, periodic retraining, runtime monitoring
Chaos Mitigation	Error validation, logging, threshold control, rollback mechanisms

3. Proposed System Architecture

3.1. Overview

The following section presents the architectural design of the Grupo Bimbo Demand Forecasting System, integrating best practices from the field of data engineering, machine learning systems, and lessons learned from the winning solution of the Kaggle competition. The architecture is designed to support a **high-volume, data-driven forecasting pipeline**, capable of generating **weekly demand predictions** using historical sales data and product/client metadata.

Informed by the analysis of the top-ranked solution, this architecture is structured to maximize **predictive performance**, **system scalability**, and **operational reliability**. The design incorporates a **multi-level machine learning ensemble**, **dynamic feature engineering with lagged variables**, and **cloud-based computing resources** to meet the computational demands of the solution.

3.2. Architectural Components Description

3.2.1. Global Configuration Management Layer

The system architecture begins with a **Global Configuration Management Layer** that centralizes all execution parameters. This includes:

- File paths for input datasets.
- Hyperparameters for machine learning models.
- Feature selection criteria, such as the number of top-ranked features to include.
- Lag configurations
- Random seeds to ensure reproducibility.

This layer ensures that all modules in the pipeline operate consistently under a shared configuration framework. By externalizing these settings, the system achieves high **maintainability** and **experiment reproducibility**, enabling researchers and engineers to re-run experiments under controlled and comparable conditions without modifying core code.

3.2.2. Data Ingestion Module

The **Data Ingestion Module** serves as the system's data acquisition entry point. It is responsible for:

- Loading large-scale transactional data containing weekly sales, client IDs, product IDs, and sales channels.
- Integrating auxiliary data sources, such as product descriptions and client segmentation metadata.
- Validating the schema to ensure data consistency.

Key joins are performed using identifiers such as **Producto_ID**, **Cliente_ID**, and **Agencia_ID**, merging transactional data with metadata to form a **unified analytical dataset**. This modular ingestion design enables the system to **adapt to new data sources or schema changes** with minimal adjustments, supporting the system's **scalability and extensibility**.

3.2.3. Advanced Data Enrichment and Feature Engineering Module

Recognizing that **feature quality directly impacts model performance**, this module executes **extensive feature engineering** steps inspired by the competition's top solution:

- **Text Parsing and Product Clustering**: Product names are parsed to extract meaningful attributes such as weight, brand, and category. Products are clustered based on these attributes, adding categorical cluster features.
- **Lagged Feature Generation**: Rolling statistical aggregations (mean, min, max, sum) are computed over the previous 1 to 3 weeks of sales data. This models real-world ordering behavior, where unsold stock influences future demand.
- **Frequency Features**: The system computes frequency counts of various factor combinations, such as client-product-channel interactions, providing insights into recurring transactional patterns.
- **Feature Selection**: Using XGBoost's feature importance function, the system ranks and selects the top features, typically retaining the **top 175** most predictive variables. This step reduces dimensionality while preserving predictive power.

These enrichment steps transform raw data into a **highly informative feature matrix**, positioning the system to achieve **high predictive accuracy**.

3.2.4. Multi-Level Model Training and Validation Module

The core innovation of this system is its **multi-level ensembling strategy**, designed to **combine the strengths of multiple modeling approaches**:

- **First-Level Models**: Multiple **XGBoost models** are trained on different feature sets and configurations. These models individually capture different aspects of the demand patterns.
- **Validation Framework**: The system simulates **real-world prediction scenarios** by first predicting one week's sales, using those predictions to calculate lagged features for the next week's forecasts. This **iterative validation process** ensures the model's performance reflects its deployment behavior.
- **Second-Level Models**: Predictions from the first-level models are used as input to **second-level models**, including:
    - **ExtraTreesClassifier**: A tree-based ensemble that captures non-linear relationships.
    - **Linear Regression Model**: A simple model that acts as a stabilizer in the ensemble, reducing the risk of overfitting.
- **Weighted Averaging**: The final forecast is generated by computing a **weighted average** of the second-level model predictions, balancing the strengths of both models.

This **stacked generalization** strategy improves generalization by reducing the variance associated with any single model, enhancing the system's **robustness and reliability**.

3.2.5. Iterative Prediction Engine

Real-world demand forecasting requires sequential decision-making. The **Iterative Prediction Engine** mimics this by:

- Predicting sales for one week (e.g., week 10).
- Using those predictions to compute new lagged features.
- Repeating the process for subsequent weeks (e.g., week 11).

This **stepwise forecasting process** reflects the operational reality where decisions made today affect outcomes tomorrow. It also introduces controlled **chaos management**, by iteratively recalculating features based on model outputs, preventing uncontrolled error propagation across forecasting horizons.

3.2.6. Output Formatting and Submission Module

Once predictions are generated, the **Output Formatting Module**:

- Formats the forecasts according to **Kaggle's submission guidelines**.
- Packages results into a **submission-ready CSV file**.

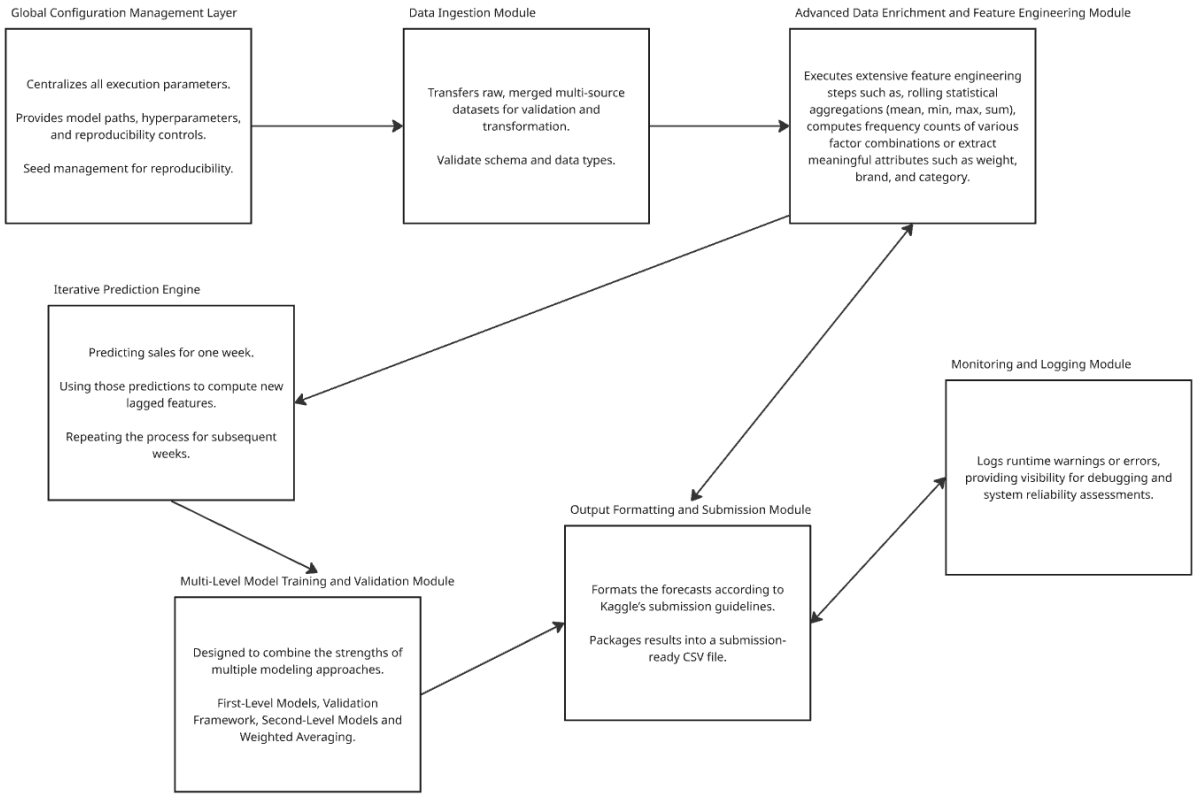
This ensures **operational readiness**, allowing for direct submission to Kaggle or integration into business workflows without additional processing.

3.2.7. Monitoring and Logging Module

To ensure **observability and system health tracking**, this module:

- Captures **execution logs**, including data loading times, feature generation statistics, and model training durations.
- Records **model evaluation metrics**, such as validation RMSLE and feature importance rankings.
- Logs **runtime warnings or errors**, providing visibility for debugging and system reliability assessments.

These capabilities are essential for maintaining **operational reliability**, especially when the system scales to cloud environments or large datasets.



4. Sensitivity and Chaos Management

4.1. Context and Importance

In complex, high-volume data systems like Grupo Bimbo's demand forecasting pipeline, **small variations in input data or system behavior** can lead to **amplified errors** or **model instability**. These characteristics reflect the principles of **chaotic systems**, where the outcomes are highly sensitive to initial conditions, unexpected variations, or feedback loops.

The proposed system is designed not only to deliver predictive performance but also to maintain **stability and reliability** in the face of such chaotic factors. This section describes the architectural and procedural mechanisms built into the system to handle these challenges, ensuring the forecasting process remains trustworthy, robust, and operationally safe.

4.2. Identification of High-Sensitivity Factors

Through the analysis of both the competition's dataset and the winning solution, several **high-sensitivity factors** were identified:

1. **Historical Demand Lags**
    a. The use of lagged demand features (1-3 weeks) introduces **feedback loops** where prediction errors can propagate across sequential forecasting periods if not properly controlled.
2. **Sparse and Noisy Data**
    a. Some client-product combinations appear **infrequently**, leading to **unstable statistical estimates** and **overfitting risks** when the model encounters underrepresented patterns.
3. **Outliers and Abnormal Sales Patterns**
    a. Unusual spikes or drops in sales may distort rolling statistics or model training, especially in **edge cases** not well-represented in the training data.
4. **Overfitting Due to High Feature Dimensionality**
    a. The generation of **hundreds of engineered features** introduces the risk of **overfitting to noise** rather than generalizable patterns.
5. **Sequential Prediction Drift**
    a. The iterative nature of forecasting multiple weeks, where predictions are fed back as new features, can lead to **cumulative error drift** if not carefully managed.

4.3. Design Strategies to Manage Sensitivity and Chaos

4.3.1. Controlled Feature Engineering

To mitigate noise and overfitting risks:

- **Feature Selection**: The system uses `xgb.importance()` to select the top 175 features, balancing model complexity and stability.
- **Aggregation Strategies**: Rolling means, min, max, and sum are computed over **controlled time windows**, smoothing out short-term volatility.
- **Product Clustering**: Grouping products into clusters reduces sparsity and improves the model's ability to generalize from similar product behaviors.

4.3.2. Sequential Validation and Feedback Loop Control

To address feedback loop sensitivity:

- **Stepwise Forecasting Process**: The system simulates real operational usage by first predicting one week's demand, recalculating lagged features based on predictions, and repeating the process. This **validates the model's stability** over consecutive periods.

- **Blended Ensemble Models**: By combining diverse models (XGBoost, ExtraTrees, Linear Regression), the system mitigates the risk of **single-model instability** dominating the forecasts.

4.3.3. Noise and Outlier Handling

To reduce the impact of abnormal data:

- **Data Cleaning**: The preprocessing stage removes or flags clearly invalid entries (e.g., negative sales values).
- **Smoothing Techniques**: Rolling statistical features reduce the impact of isolated spikes or drops in demand.
- **Frequency-Based Filtering**: Rare combinations of client-product data are handled with caution, preventing unstable features from dominating the model.

4.3.4. Model Regularization and Cross-Validation

To prevent overfitting and improve generalization:

- **XGBoost Regularization Parameters**: Parameters such as `gamma`, `lambda`, and `alpha` are tuned to control model complexity.
- **Cross-Validation Framework**: The model undergoes **multi-week validation simulations** to ensure that performance generalizes across different time periods.

4.4. Monitoring and Error-Handling Routines

4.4.1. Runtime Monitoring

The system implements **real-time logging** to capture:

- **Data Statistics**: Distribution summaries of key variables during ingestion and preprocessing.
- **Model Metrics**: Validation scores, feature importances, and prediction distributions at each stage.
- **Execution Metadata**: Timestamps, runtime durations, and system resource usage.

This ensures **observability** and supports **post-run diagnostics**.

4.4.2. Error Detection and Reporting

To handle unanticipated conditions:

- **Schema Validation Errors**: Automatically stop execution if expected data columns or formats are missing.
- **NaN or Infinite Value Checks**: Detect and handle mathematical anomalies in feature computations.
- **Prediction Range Validation**: Ensure that predicted demand values are **non-negative and realistic**, rejecting out-of-bound outputs.
- **Exception Handling**: Structured try-catch blocks capture runtime exceptions, log detailed error messages, and safely terminate or retry the process.

4.5. Summary of Sensitivity and Chaos Management Strategies

Sensitivity/Chaos Factor	Mitigation Strategy
Feedback Loops in Lagged Features	Iterative validation and controlled lag recalculation.
Sparse and Noisy Data	Aggregation, product clustering, and frequency-based feature filtering.
Outliers and Abnormal Patterns	Rolling statistical features and data cleaning.
High Dimensionality	Feature selection via importance ranking to retain only top-performing features.
Sequential Prediction Drift	Stepwise forecasting with recalculated lags based on predicted outputs.
Runtime Errors or Anomalies	Schema checks, NaN detection, prediction validation, and exception handling.
Observability Requirements	Comprehensive runtime logging of data properties, model performance, and system health.
Cloud Execution Reliability	Parallel processing, distributed computation, and job recovery mechanisms in cloud environments.

5. Technical Stack and Implementation Sketch

5.1. Introduction

Successfully implementing the proposed architecture requires a **production-ready technical stack** capable of processing large volumes of structured data, engineering complex features, training robust machine learning models, and generating forecasts efficiently. Based on scalability, industry adoption, and community support, **Python** is selected as the **sole programming language** for the entire pipeline. This decision ensures a **unified development environment**, **easier maintenance**, and **faster onboarding** for future contributors.

5.2. Technical Stack Selection

5.2.1. Programming Language

- **Python (Primary and Exclusive)**
    - Industry-standard language for data science and machine learning.
    - Rich ecosystem of libraries supporting the entire machine learning lifecycle.
    - Active community and abundant resources for troubleshooting and learning.
    - Supports both **research and production deployment** seamlessly.

Rationale: Focusing entirely on Python ensures **consistency**, **reduced complexity**, and **full integration** across all pipeline stages.

5.2.2. Key Libraries and Frameworks

Data Processing and Manipulation

- **Pandas**: High-performance data manipulation, merging, and transformation.
- **NumPy**: Efficient numerical computing for large-scale array operations.

Feature Engineering and Statistical Computation

- **SciPy**: Advanced statistical methods for feature construction.

- **nltk / re**: Text parsing and regular expressions for extracting product metadata (e.g., weight, brand).
- **scikit-learn (feature_extraction)**: Tools for categorical encoding and frequency-based features.

Machine Learning and Model Evaluation

- **XGBoost**: High-performance gradient boosting framework for first-level model training.
- **scikit-learn**: Provides ExtraTreesClassifier, Linear Regression, cross-validation utilities, and evaluation metrics.

Experimentation and Interactive Development

- **Jupyter Notebooks**: Interactive Python environment for prototyping, testing, and documenting workflows.

Configuration Management

- **PyYAML / json**: Libraries for parsing and managing external configuration files.

Logging and Monitoring

- **Python logging module**: Standardized logging of execution metrics, runtime statistics, and error tracking.

5.3. Computing Environment

5.3.1. Local Development Environment

- Python 3.x with all required packages managed through **virtual environments** (e.g., venv or conda).
- Jupyter Notebooks for iterative exploration, feature engineering, and documentation.

5.4. Implementation Sketch

5.4.1. Stage 1: Data Ingestion

- Load transactional and metadata CSV files using **Pandas**.

- Validate data types, column presence, and key relationships.
- Log schema validation results.

5.4.2. Stage 2: Data Preprocessing

- Handle missing values and outliers.
- Deduplicate records.
- Aggregate data to the **client-product-week level**.

5.4.3. Stage 3: Feature Engineering

- **Text Parsing**: Use `re` and `nltk` to extract product attributes (brand, weight).
- **Rolling Statistics**: Compute lagged demand aggregates over **1–3 week windows** using **Pandas** and **NumPy**.
- **Frequency-Based Features**: Generate interaction counts with **scikit-learn feature extraction tools**.
- **Median-Based Fallbacks**: Apply **groupby().median()** in Pandas for low-frequency combinations.
- **Feature Selection**: Rank and filter top **175 features** using **XGBoost's feature importance tools**.

5.4.4. Stage 4: First-Level Model Training

- Train multiple **XGBoost regressors** with varied feature subsets and configurations.
- Store predictions and model metadata.

5.4.5. Stage 5: Second-Level Model Training

- Train **ExtraTreesClassifier** and **Linear Regression models** using **scikit-learn**.
- Evaluate via cross-validation.

5.4.6. Stage 6: Iterative Prediction Engine

- Generate sequential forecasts, updating lagged features with model outputs for multi-week predictions.

5.4.7. Stage 7: Output Formatting

- Generate **Kaggle submission CSV** files.
- Validate predictions for compliance and non-negativity.

5.4.8. Stage 8: Monitoring and Logging

- Log all runtime metrics, including data characteristics, model scores, and execution durations.
- Capture and report errors or warnings.

5.5. Summary

Category	Tools / Frameworks
Language	Python (Primary and Exclusive)
Data Processing	Pandas, NumPy
Feature Engineering	SciPy, nltk, re, scikit-learn (feature_extraction)
Machine Learning	XGBoost, scikit-learn (ExtraTreesClassifier, Linear Regression)
Interactive Development	Jupyter Notebooks
Configuration Management	PyYAML, json
Logging and Monitoring	Python logging module
Version Control	Git, GitHub
Documentation	Jupyter Notebooks, Markdown README